# UNIVERSIDAD NACIONAL DE UCAYALI

## FACULTAD DE INGENIERÍA DE SISTEMAS E INGENIERÍA CIVIL

### ESCUELA DE PROFESIONAL DE INGENIERÍA DE SISTEMAS

# CONCEPTOS AVANZADOS RELACIONADOS A POO

***PROGRAMACIÓN ORIENTADA A OBJETOS II***

**Docente : Ing. M.Sc. Edson Greig Cáceres Cárdenas**

**Pucallpa – Perú**

**2026**

# CLASES

Una clase es la unidad básica de la Programación Orientada a Objetos (POO). Es el "molde" a partir del cual se crean los objetos.

- **Definición**: Define el estado (atributos) y el comportamiento (métodos) que compartirán los objetos de ese tipo.
- **Características principales:**
  1. Pueden ser instanciadas (puedes crear un objeto con new).
  2. Pueden contener métodos con implementación completa.
  3. Soportan modificadores de acceso (public, private, protected).

```java
package modifier;

public class User extends Person {
    private String username;
    private String password;

    @Override
    public String getName(){
        return this.name;
    }
}
```

ini modifier

# CLASES ABSTRACTAS

Una clase abstracta es una clase que **no se puede instanciar**. Se utiliza para definir una base común para una jerarquía de clases.

**Definición:** Se declara con la palabra reservada *abstract*. Su propósito es ser heredada (extendida).

**Características principales:**

- **Métodos Abstractos:** Puede tener métodos sin cuerpo (solo la firma) que las subclases **deben** implementar.
- **Métodos Concretos:** También puede tener métodos con lógica ya definida.
- **Estado:** Puede tener variables de instancia (atributos) y constructores.

**Teoría:** Se usa cuando tienes una relación de "Es un" (**Is-a**). Por ejemplo, Animal puede ser una clase abstracta, ya que no existe un "animal" genérico caminando por ahí, sino perros o gatos específicos que heredan de él.

```java
public abstract class GenericKafkaConsumerConfig {

    @Value("${kafka.peruda.server}")
    private String kafkaServer;

    @Value("${kafka.peruda.port}")
    private String kafkaPort;

    @Value("${kafka.peruda.groupid}")
    private String groupId;

    @Value("${kafka.peruda.trusted-packages}")
    private String trustedPackages;

    protected ConsumerFactory<String, KafkaMessage<?>> consumerFactory() {
        Map<String, Object> kafkaProperties = new HashMap<>();
        kafkaProperties.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, kafkaServer + ":" + kafkaPort);
        kafkaProperties.put(ConsumerConfig.GROUP_ID_CONFIG, groupId);

        kafkaProperties.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, ErrorHandlingDeserializer.class);
        kafkaProperties.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, ErrorHandlingDeserializer.class);

        kafkaProperties.put(ErrorHandlingDeserializer.KEY_DESERIALIZER_CLASS, JsonDeserializer.class);
        kafkaProperties.put(ErrorHandlingDeserializer.VALUE_DESERIALIZER_CLASS, JsonDeserializer.class.getName());

        kafkaProperties.put(JsonDeserializer.TRUSTED_PACKAGES, trustedPackages);

        return new DefaultKafkaConsumerFactory<>(kafkaProperties);
    }

    @Bean
    protected ConcurrentKafkaListenerContainerFactory<String, KafkaMessage<?>> kafkaListenerContainerFactory() {
        ConcurrentKafkaListenerContainerFactory<String, KafkaMessage<?>> factory = new ConcurrentKafkaListenerContainerFactory<>();
        factory.setConsumerFactory(this.consumerFactory());
        return factory;
    }

    protected abstract ObjectMapper getMapper();

    protected abstract <T> IGenericRepository<T, String> getRepo(T t) throws Exception;
```

# CLASES ABSTRACTAS

```java
public abstract class GenericKafkaConsumerConfig {

    @Value("${kafka.peruda.server}")
    private String kafkaServer;

    @Value("${kafka.peruda.port}")
    private String kafkaPort;

    @Value("${kafka.peruda.groupid}")
    private String groupId;

    @Value("${kafka.peruda.trusted-packages}")
    private String trustedPackages;

    protected ConsumerFactory<String, KafkaMessage<?>> consumerFactory() {
        Map<String, Object> kafkaProperties = new HashMap<>();
        kafkaProperties.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, kafkaServer + ":" + kafkaPort);
        kafkaProperties.put(ConsumerConfig.GROUP_ID_CONFIG, groupId);

        kafkaProperties.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, ErrorHandlingDeserializer.class);
        kafkaProperties.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, ErrorHandlingDeserializer.class);

        kafkaProperties.put(ErrorHandlingDeserializer.KEY_DESERIALIZER_CLASS, JsonDeserializer.class);
        kafkaProperties.put(ErrorHandlingDeserializer.VALUE_DESERIALIZER_CLASS, JsonDeserializer.class.getName());

        kafkaProperties.put(JsonDeserializer.TRUSTED_PACKAGES, trustedPackages);

        return new DefaultKafkaConsumerFactory<>(kafkaProperties);
    }

    @Bean
    protected ConcurrentKafkaListenerContainerFactory<String, KafkaMessage<?>> kafkaListenerContainerFactory() {
        ConcurrentKafkaListenerContainerFactory<String, KafkaMessage<?>> factory = new ConcurrentKafkaListenerContainerFactory<>();
        factory.setConsumerFactory(this.consumerFactory());
        return factory;
    }

    protected abstract ObjectMapper getMapper();

    protected abstract <T> IGenericRepository<T, String> getRepo(T t) throws Exception;
```

```java
public class KafkaConsumerConfig extends GenericKafkaConsumerConfig{

    //No need to use service 'cause everything is done by service previously
    private final IPlaceRepository placeRepository;
    private final IReasonRepository reasonRepository;
    private final IUserRepository userRepository;

    private final ObjectMapper objectMapper = JsonMapper.builder().addModule(new JavaTimeModule()).build();

    @SuppressWarnings("unused")
    private final TopicUtil topicUtil;

    @Override
    protected ObjectMapper getMapper() {
        return objectMapper;
    }

    @SuppressWarnings("unchecked")
    @Override
    protected <T> IGenericRepository<T, String> getRepo(T t) throws Exception {
        return switch (t) {
            case Place p -> (IGenericRepository<T, String>) this.placeRepository;
            case Reason r -> (IGenericRepository<T, String>) this.reasonRepository;
            case User u -> (IGenericRepository<T, String>) this.userRepository;
            default -> throw new Exception("Not value found for repository.");
        };
    }

}
```

# INTERFACES

Una interfaz es una referencia que define **qué** debe hacer una clase, pero no **cómo** lo hace. Es un contrato puramente formal.

**Definición:** Se define con la palabra interface. Una clase "implementa" una interfaz.

**Características principales:**

- **Abstracción Total:** Tradicionalmente, todos sus métodos son abstractos.
- **Herencia Múltiple:** Java no permite heredar de varias clases, pero una clase puede implementar **múltiples interfaces**.
- **Constantes:** Todas las variables declaradas son implícitamente public static final.

**Teoría:** Representa una capacidad o rol (**Can-do**). Por ejemplo, la interfaz *Volador*. No importa si eres un Avión o un Pájaro, si implementas *Volador*, debes tener el método despegar().

```java
public interface ICrudDao<T extends BEntity> {
    public long count();

    public void delete(Long id);

    public List<T> findAll();

    public T findById(Long id);

    public T findByName(String propertyName, Object value);

    public List<T> findDeleted();

    public T save(T t);

    public T update(T t);
}
```

# COMPARATIVA

| Característica | Clase Concreta | Clase Abstracta | Interfaz |
|---|---|---|---|
| **Instanciable** | Sí | No | No |
| **Herencia** | Una sola clase | Una sola clase | Múltiples interfaces |
| **Métodos** | Solo implementados | Mezcla de ambos | Mayormente abstractos |
| **Atributos** | Cualquier tipo | Cualquier tipo | Solo constantes ( `final` ) |

# PROGRAMACIÓN GENÉRICA

La programación genérica permite que las clases, interfaces y métodos operen sobre tipos de datos (clases) que se especifican como **parámetros** al momento de ser utilizados.

**El concepto de "Tipo Parametrizado"**

En lugar de escribir una clase para manejar Integers y otra idéntica para Strings, escribes una sola estructura utilizando un marcador de posición, generalmente representado por letras como <T> (Type), <E> (Element) o <K, V> (Key, Value).

**¿Por qué son fundamentales?**

1. **Seguridad de Tipos (Type Safety):** El compilador verifica que estás usando el tipo correcto. Antes de los genéricos (Java 1.5), se usaba la clase Object, lo que obligaba a hacer "castings" manuales que solían fallar en tiempo de ejecución.
2. **Eliminación de Castings:** No necesitas convertir manualmente el objeto al extraerlo de una colección.
3. **Reutilización de Código:** Un solo algoritmo puede aplicarse a diferentes tipos de datos sin duplicar lógica.

# PROGRAMACIÓN GENÉRICA

```java
public abstract class CRUDServiceKafkaProducerImpl<T, ID> extends CRUDServiceImpl<T, ID> {

    protected abstract KafkaUtil getKafkaUtil();

    protected abstract List<String> getTopics();

    @SuppressWarnings("unchecked")
    protected void sendKafkaMessage(String operation, LocalDateTime date, T t) {
        KafkaMessage<Object> kafkaMsg = KafkaMessage.builder()
            .typeOperation(operation)
            .date(date)
            .contentClassType((Class<Object>) t.getClass())
            .content(t)
            .build();
        getKafkaUtil().sendMessage(getTopics(), kafkaMsg);
    }
```

GRACIAS